EMBEDDED SYSTEMS & INTERNET OF THINGS (IoT)

TASK 1 — COMPLETED ASSIGNMENT

Topic: Fundamentals of Embedded Systems and Design of a Smart Sensor System

Prepared as: Junior Embedded Engineer — Mini Project Report

Project selected: Motion Detection Alarm using an Arduino-compatible controller, PIR motion sensor, LED and buzzer

Based on the uploaded task sheet. Where the sheet leaves choices open, this report states the assumptions made.

Contents

1. Introduction and research

2. Component study

3. Mini project: Motion Detection Alarm

4. Circuit diagram and connections

5. Arduino/C++ program and explanation

6. Testing, limitations and improvements

7. Real-world applications

8. Conclusion

9. Information not specified in the task sheet

1. Introduction and Research

1.1 What is an embedded system?

An **embedded system** is a computer-based electronic system built into a larger product to perform a specific function or a small set of related functions. It combines hardware—such as a processor, memory, input/output circuits and sensors—with software or firmware designed for its intended task. Unlike a general-purpose computer, it is usually designed around a defined job, power budget, cost and response time.

Typical features

• Dedicated purpose: it is designed for a particular function.
• Inputs and outputs: it reads switches or sensors and controls displays, motors, LEDs, relays or other devices.
• Firmware: software stored in memory controls its operation.
• Resource constraints: many systems have limited memory, processing power and energy.
• Predictable response: control systems often need to react within a known time.

1.2 Real-world applications

Area	Examples	Purpose
Smart home	Thermostats, smart lights, door sensors	Automation, comfort, energy saving and security
Automotive	Engine control unit, ABS, airbag controller, parking sensors	Vehicle control, safety and diagnostics
Healthcare	Digital thermometer, pulse oximeter, infusion pump	Measurement, monitoring and controlled delivery
Robotics	Motor controller, line-following robot, obstacle detection	Sensing, movement and feedback control
Industry	PLC, temperature controller, condition-monitoring node	Automation, quality control and equipment protection
Consumer devices	Washing machine controller, printer, camera	Coordinating user commands and internal operations

1.3 Microcontroller vs microprocessor

Feature	Microcontroller (MCU)	Microprocessor (MPU)
Integration	Usually includes CPU, RAM, flash and peripherals on one chip	Often relies on external memory and support chips; modern SoCs are an exc
Typical use	Dedicated control, sensing and embedded tasks	General-purpose computing and more demanding operating systems
Power/cost	Often low power and low cost for control tasks	Often higher processing capability and system complexity
Peripherals	GPIO, timers, ADC, serial interfaces commonly integrated	May use external or integrated peripherals depending on design
Examples	ATmega328P, ESP32, RP2040	Processors used in PCs and application-class computing boards

In short, a microcontroller is generally a compact control computer with many on-chip peripherals. A microprocessor generally emphasizes computation and may need more external components, although modern chips can combine both approaches.

2. Component Study

2.1 Microcontrollers and development boards

Arduino UNO: A beginner-friendly development board commonly built around the ATmega328P (classic UNO R3). It provides digital input/output pins, analog inputs, timers and serial communication. It is suitable for learning, simple sensor projects and basic control. The exact chip and capabilities vary across UNO versions.

ESP32: A family of microcontrollers with integrated Wi-Fi and Bluetooth on many variants. It is useful for connected sensors, remote monitoring, smart-home nodes and IoT prototypes. Pin assignments, voltage limits and radio features depend on the specific ESP32 board.

Raspberry Pi Pico: A microcontroller development board based on the RP2040 in the original Pico. It offers GPIO, PWM, ADC and serial interfaces and can be programmed in C/C++ or MicroPython. The original Pico does not include Wi-Fi; Pico W variants add wireless connectivity.

These are development boards rather than interchangeable parts. Before wiring, check the exact board pinout, logic voltage, current limits and programming environment.

2.2 Basic sensors

Temperature sensor: Measures temperature. Examples include the analog LM35, digital DS18B20 and temperature sensors built into some modules. The controller reads the sensor output and converts it into a temperature value. The chosen sensor determines the wiring and conversion method.

Motion sensor: A passive infrared (PIR) module detects changes in infrared energy, commonly caused by a warm body moving through its field of view. Many modules provide a digital HIGH signal when motion is detected. PIR sensors detect changes rather than identify a person, and performance depends on placement and settings.

Light sensor: A light-dependent resistor (LDR) changes resistance with illumination. With a fixed resistor it can form a voltage divider whose output is read by an analog input. Photodiodes and digital ambient-light sensors are alternatives.

2.3 Other components used in this project

LED: A visual indicator. A series resistor limits current and protects the LED and controller pin.
Buzzer: Produces an audible alert. Use a suitable active buzzer for a simple HIGH/LOW control example; check its current requirement.
Resistor: Limits current or sets an electrical level. A 220–330 Ω resistor is a common starting range for a small indicator LED on a 5 V Arduino UNO, subject to the LED and board specifications.
Breadboard and jumper wires: Allow a circuit to be assembled without soldering.

3. Mini Project — Motion Detection Alarm

3.1 Objective

Build a simple alarm that detects movement using a PIR sensor and activates an LED and buzzer. The prototype is intended for learning and demonstration; it is not a certified security alarm.

3.2 Working principle

1. The PIR module monitors changes in infrared energy in its field of view.
2. When it detects movement, its digital output becomes HIGH (depending on the module's configuration).
3. The microcontroller reads that input.
4. If the input is HIGH, the program switches on the LED and buzzer.
5. If the input is LOW, the program switches both outputs off.

Block diagram

PIR Motion Sensor Digital signal	**Arduino-compatible MCU** Read input and apply logic	**LED + Buzzer** Visual and audible alert

Signal flow: motion → PIR output → controller decision → LED/buzzer alert.

3.3 Assumptions

The task sheet permits a real circuit or a simulation but does not specify a board, sensor model, pin numbers or alarm timing. This report assumes a classic 5 V Arduino UNO, a common 3-pin digital-output PIR module, an LED with a current-limiting resistor, and a compatible active buzzer. The code uses example pin assignments that can be changed to match the simulator or hardware.

4. Circuit Diagram and Connections

Use the following wiring table as the circuit plan. Connect all grounds together. Do not connect a buzzer that exceeds the board pin's current rating directly to a GPIO; use a transistor driver and suitable supply if required.

Component pin	Connect to	Notes
PIR VCC	Arduino 5V	Use the voltage required by the exact PIR module; some modules support a wi
PIR GND	Arduino GND	Common ground is required.
PIR OUT	Arduino D2	Digital input.
Arduino D8	220–330 Ω resistor → LED anode (+)	LED cathode (–) to GND.
Arduino D9	Compatible active buzzer (+)	Buzzer (–) to GND only if within pin-current limits; otherwise use a driver stage.

Text circuit diagram

```
                  +--------------------+
                  |     Arduino UNO    |
PIR OUT -------->| D2             D8  |----[220-330 ohm]---->| LED |---- GND
                  |                D9  |----(+) BUZZER (-)---- GND*
PIR VCC -------->| 5V                 |
PIR GND -------->| GND                |
                  +--------------------+

*Only connect the buzzer directly if its current is within the board pin rating.
Otherwise use a transistor/MOSFET driver and a suitable supply.
All circuit grounds must be connected together.
```

Figure 1. Simplified connection diagram (not to scale). Verify the pinout and electrical ratings of the exact components before powering the circuit.

5. Arduino/C++ Program and Explanation

5.1 Code

```
const int PIR_PIN = 2;
const int LED_PIN = 8;
const int BUZZER_PIN = 9;

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int motionState = digitalRead(PIR_PIN);

  if (motionState == HIGH) {
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
    Serial.println("Motion detected! Alarm ON");
  } else {
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
    Serial.println("No motion. Alarm OFF");
  }

  delay(200);
}
```

5.2 Explanation of the program

• Pin constants: PIR_PIN, LED_PIN and BUZZER_PIN name the selected input and output pins. Change them if your wiring differs.

• setup(): runs once when the board starts. It configures the PIR pin as an input, the LED and buzzer as outputs, and

starts serial communication at 9600 baud.
• digitalRead(): reads the PIR module's digital output as HIGH or LOW.
• if/else: when the reading is HIGH, the program turns on the alert outputs; otherwise, it turns them off.
• delay(200): waits 200 milliseconds between readings. The PIR module may hold its output HIGH for a configured period, so the alert can remain on after movement stops.

5.3 Example output

When the sensor detects motion, the Serial Monitor may display “Motion detected! Alarm ON”. When no motion is detected, it may display “No motion. Alarm OFF”. The exact sequence depends on the PIR module's warm-up time, sensitivity, hold-time setting and surroundings.

6. Testing, Limitations and Improvements

Test	Action	Expected result
Initial power-up	Power the circuit and allow the PIR to stabilise	Sensor may produce unstable readings during warm-up; wait for the module's re
Motion present	Move across the sensor's field of view	PIR output goes HIGH; LED and buzzer activate.
No motion	Remain still/outside the field of view after the PIR output returns LOW	PIR output returns LOW; LED and buzzer switch off.
Serial check	Open Serial Monitor at 9600 baud	Text messages match the input state.

Limitations: PIR sensors do not detect all motion equally; direction, distance, temperature and obstructions matter. They can trigger falsely due to environmental changes. This simple circuit does not send notifications over the internet and is not a dependable life-safety or professional security system.

Possible improvements: add a configurable alarm duration, an arm/disarm switch, a status display, a battery supply, event logging, or an ESP32/Wi-Fi connection to send alerts. Network features require additional code, connectivity and security considerations.

7. Potential Real-World Applications

• Room or corridor occupancy indication: turn on an indicator when movement is detected.
• Basic entry alert: provide a local audible signal near a doorway or storage area.
• Energy-saving automation: use motion as one input to control lighting, ideally combined with a light sensor and a timeout.
• Robotics and interactive demonstrations: teach sensor input, digital logic and output control.
• IoT extension: connect the controller to a network-capable board such as an ESP32 to report events to a dashboard or notification service. This extension is not included in the basic code.

For real deployments, consider privacy, reliability, electrical safety, false alarms, network security and backup power. A classroom prototype should not be represented as a certified security product.

8. Conclusion

This report introduces embedded systems, compares microcontrollers and microprocessors, and describes common controllers and sensors. The proposed motion detection alarm demonstrates a complete embedded-system loop: a sensor produces an input, firmware makes a decision, and output devices provide feedback. It can be built on compatible hardware or recreated in a simulator, then extended with additional sensing or IoT connectivity.

9. Information Not Specified in the Task Sheet

The uploaded task sheet defines the assignment sections and offers three mini-project choices, but it does not specify the following details. This report makes explicit assumptions rather than presenting them as requirements from the sheet.

Not specified in source	Assumption used in this report
Which mini project to select	Motion Detection Alarm selected from the three allowed options.
Hardware board or exact model	Classic Arduino UNO / ATmega328P used for example wiring and code.
Exact sensor part number and operating voltage	Generic 3-pin digital-output PIR module assumed; check its datasheet.
Pin assignments and buzzer type	PIR D2, LED D8, buzzer D9; compatible active buzzer assumed.
Simulation platform or actual hardware	Design is hardware-oriented and can be adapted to Tinkercad or Wokwi; no simulation run is claimed.
Required report length, citation style, student details, and any tutorial	Not provided; concise report structure used and no student/institution details invented.
Whether internet connectivity is mandatory	Not mandatory for the chosen basic alarm; IoT connectivity is discussed as an optional extension.

References and learning resources listed in the task sheet

The source assignment itself lists the following resources: NPTEL Introduction to Embedded Systems; Arduino Official Getting Started Guide; Wokwi Simulator; and Tinkercad Circuits. They are listed here because they appear in the task sheet. The report has not claimed to run or verify the circuit in any simulator.

Source document: “Introduction to Embedded Systems & IoT Device Design — Embedded Systems Task-1,” uploaded by the user (2 pages).